AI DATA RELIABILITY BOT · PROPOSED ARCHITECTURE · 6 OCT 2026

Alert Bot Architecture

One bot, many projects. It starts with Slipstream and CSDW, watches the web scraping that a trigger table kicks off through Azure Data Factory, explains what went wrong with an AI agent, and alerts each project's team through Power Automate. A React portal lets people sign in and see only their own projects.

How the alert bot fits with everything else

There are three parts, and they share one database of findings. The bot is the part that does the watching. The portal and Power Automate are two ways people see what it found.

1 · THE BOT (PYTHON)

Watches and investigates

A background service that runs every 10 minutes whether or not anyone is logged in. It reads the trigger table, ADF run history and the scraped output, compares them with learned baselines, groups problems into incidents and asks the AI agent to explain them.

2 · POWER AUTOMATE

Delivers the alert

When the bot opens or escalates an incident, it calls a Power Automate flow. The flow posts a card in the project's Teams channel and emails owners for critical issues. Push only; nobody has to open anything.

3 · PORTAL (REACT + VITE)

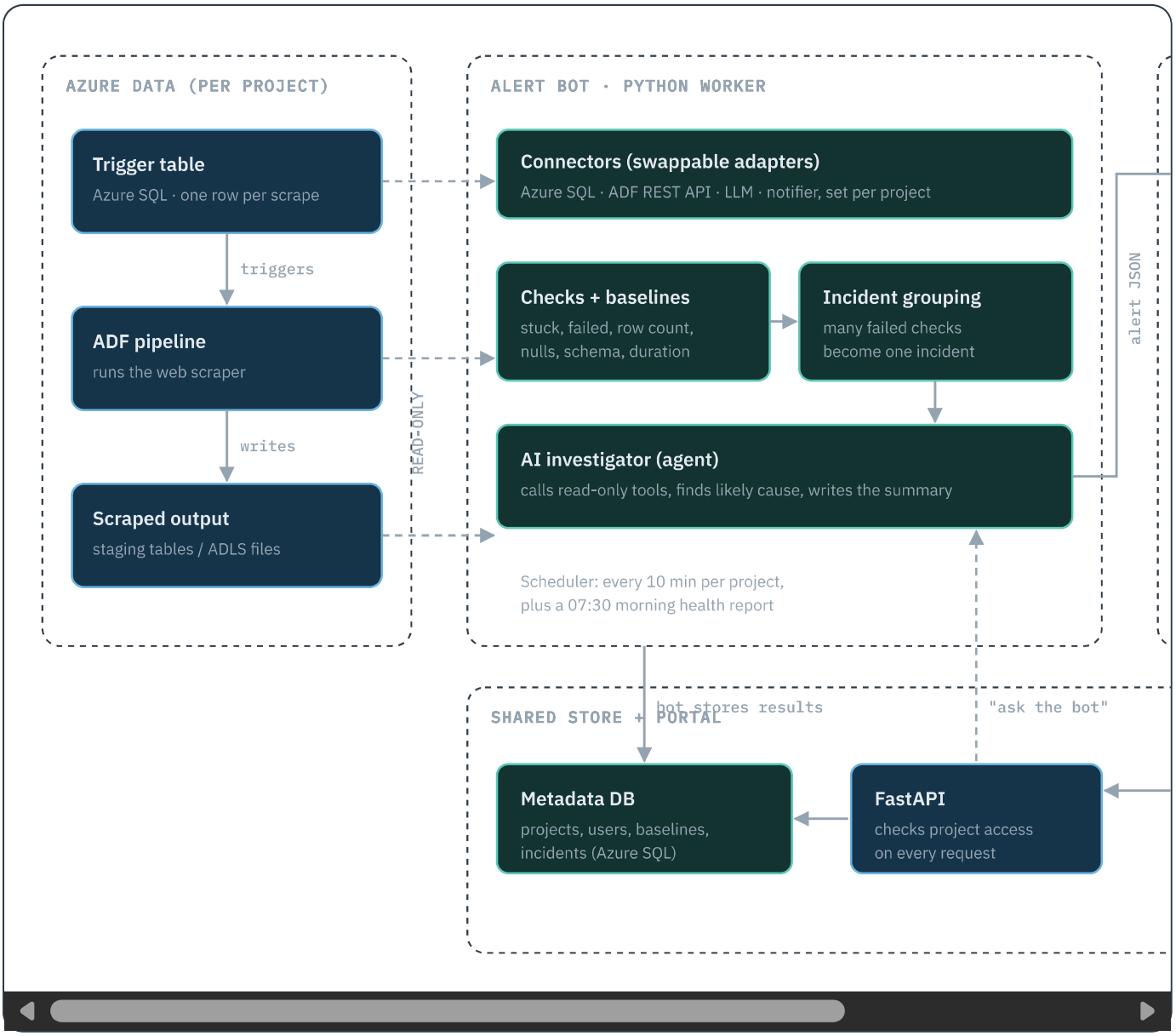
Lets people look and ask

The website. You sign in with your Microsoft account, see a tab for each project you belong to, and browse health, incidents and run history. Later you can ask the bot questions here.

> In short, the alert bot is the engine, Power Automate is its voice, and the portal is its window. The portal never runs checks itself; it reads what the bot already stored.

System diagram

System diagram



Dashed arrows into the bot are read-only access. The bot never writes to project tables or reruns pipelines in this phase.

One scrape, end to end

An example of a bad Slipstream run on a Monday. The numbers are illustrative.

06:00	**DATA** A row lands in the trigger table: site A, status `pending` .
06:01	**DATA** ADF starts the scrape pipeline and its run id is written back to the row.
06:24	**DATA** ADF reports `Succeeded` , but only 1,240 rows arrived for site A. Mondays usually bring 9,800 to 11,200.

06:30	BOT	The 10-minute cycle flags two checks: row count is 88% below baseline and `price` is null in 63% of rows (normally under 1%).
06:30	BOT	Both checks share a run and a table, so they become one incident: "Slipstream: site A scrape incomplete".
06:31	BOT	The AI agent reads the ADF activity output (HTTP 200, no errors), the schema (unchanged) and the last 7 runs (normal). Its conclusion: the site likely changed its page layout, so the price selector finds nothing.
06:31	PEOPLE	Power Automate posts the incident card in the Slipstream Teams channel and emails the owners. CSDW members get nothing.
later	PEOPLE	Someone in both projects signs in and sees Slipstream red and CSDW green, with the AI summary and evidence on the incident page.

What the bot checks on a web scraping project

Thresholds are learned from history (by weekday, per site) so nobody has to set them by hand. A project can still pin a hard rule where it wants one.

CHECK	READS FROM	CATCHES
Stuck trigger	Trigger table	Row still `pending` with no ADF run id after N minutes
Run failed	ADF REST API	Failed, cancelled or retried runs, with the failing activity and error text
Run too slow	ADF REST API	Duration far above normal; predicts a missed SLA while still running
Row count	Scraped output	Too few or too many rows per site compared with that weekday's baseline
Empty site	Scraped output	Zero rows for a site that normally returns data (blocked, 403, captcha)
Null spike	Scraped output	Key fields suddenly empty, the usual sign of a site layout change
Duplicates	Scraped output	Repeated keys, for example the same listing scraped twice
Freshness	Scraped	No new data since the last expected run

	output	
Schema drift	Scraped output	Columns added, removed or retyped
Mismatch	All three	Trigger row says success but no rows landed, or rows landed with no trigger

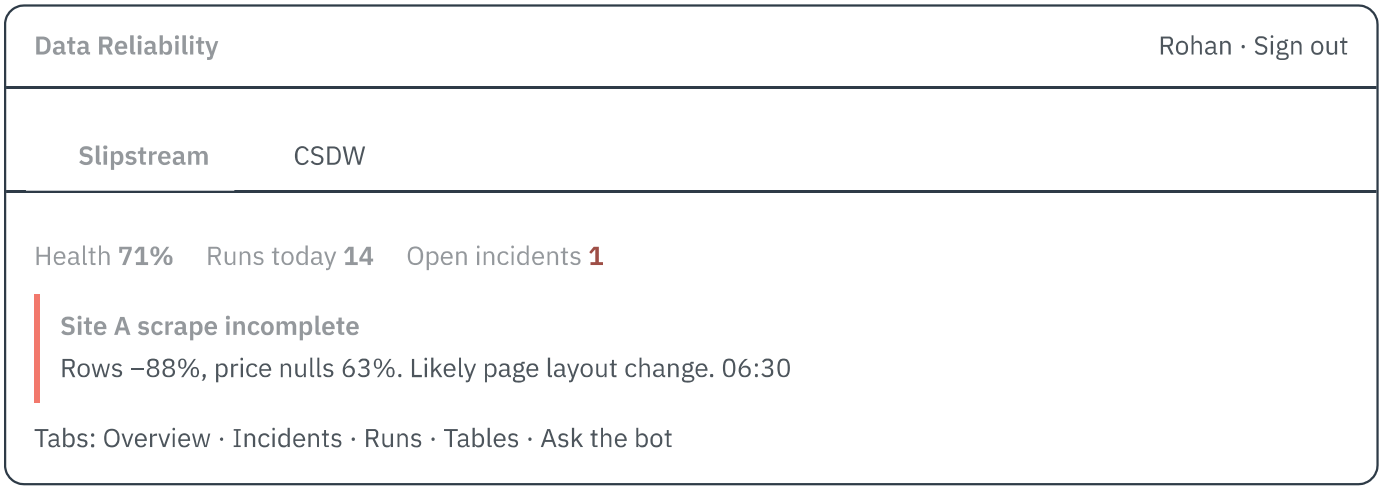
Login and project access

Users sign in with their company Microsoft account (Entra ID) through MSAL in the React app. The token goes to FastAPI, which looks the user up in a `user_projects` table and returns only their projects. Every API call checks that table again, so hiding a tab in the UI is never the only protection.

I read "windows" as a tab or panel per project inside the portal. A person in both Slipstream and CSDW sees two tabs; a person in one sees one. If you meant separate browser windows or a Windows desktop app, tell me and I'll adjust.

```
users          (id, entra_oid, email, name)
projects       (id, key, display_name, active)
user_projects  (user_id, project_id, role)
                role = viewer | owner
```

Owners can mute alerts and edit the project's config. Membership can later be filled from Entra groups instead of by hand.



Plug and play: adding a project

The bot's core knows nothing about Slipstream or CSDW. Each project is one config entry that says which connectors to use and where its tables, pipelines and alert flow are. Adding a third project

means adding a config and members; no code changes.

Each connector type is a small Python interface, so a project on Synapse, Databricks or Airflow later only needs a new adapter:

- **Data source**: run a metric query, read schema (Azure SQL first)
- **Pipeline**: list runs, get activity output (ADF first)
- **Notifier**: send an incident (Power Automate first)
- **AI provider**: chat with tools (whichever LLM is approved)

Secrets stay in Azure Key Vault; the config only names them. The bot uses a managed identity with read-only roles.

```
project: slipstream
display_name: Slipstream
schedule: "*/10 * * * *"

sources:
  - id: main
    type: azure_sql
    database: slipstream
    auth: managed_identity

pipeline:
  type: adf
  resource_group: rg-slipstream
  factory: adf-slipstream
  pipelines: [pl_web_scrape]

trigger_table:
  source: main
  table: ctl.scrape_requests
  status_column: status
  run_id_column: adf_run_id
  created_column: created_at

outputs:
  - table: stg.scraped_listings
    key: [site, listing_id]
    group_by: site
    loaded_at: loaded_at

alerts:
  notifier: power_automate
  flow_url: kv://pa-slipstream-alerts
  teams_channel: Slipstream Data Alerts
```

The agentic AI part

Checks and grouping are plain Python, so detection works even if the AI is down. The agents are built with LangGraph. LangChain is used only for the model connector and tool definitions, and everything sits behind the bot's AI adapter so the framework or model can be swapped later.

AGENT	WHEN IT RUNS	WHAT IT DOES
Investigator	Each new or worsening incident	Picks which read-only tools to call, finds the likely cause and returns what happened, impact, suggested action and evidence
Ask the bot	Phase 2, from the portal	Answers questions with the same tools, limited to the signed-in user's projects
Morning report	07:30 daily	A single LLM call that summarises the day's results. It is not an agent.

Each agent gets the incident or question plus the read-only tools below:

TOOL	WHAT IT RETURNS
`get_trigger_rows`	Trigger rows for a site or time window, with status and run ids
`get_adf_run`	Pipeline run status, duration and parameters
`get_activity_runs`	Each activity's output and error text (HTTP codes, rows copied)
`query_metric`	Row count, null rate or distinct count for a table and window
`get_baseline`	The learned normal range for a metric on this weekday
`compare_schema`	Column differences against the last known schema
`recent_incidents`	Similar past incidents for this project and how they ended

Guardrails: tools only run fixed, parameterised queries scoped to the incident's project; the agent cannot write SQL of its own or touch another project; every tool call is logged with the incident.

Model: not chosen yet

The model decision is on hold. The bot has one shared AI setting with empty slots for the provider, model, endpoint and key. The key lives in Key Vault under a fixed name, so choosing a model later only means filling in this block and storing the key. No code changes are needed.

Until a key is set, the bot still runs every check and sends alerts. Incidents just carry a rule-based summary (metric, expected range, actual value) instead of the AI explanation.

```
# config/ai.yaml  (shared by all projects)
```

```
# config/ai.yaml  (shared by all projects)
ai:
  enabled: false        # flip to true once a key is in
  provider: ""          # to be decided
  model: ""             # to be decided
  endpoint: ""          # to be decided
  api_key: kv://alert-bot-llm-api-key
                        # placeholder secret in Key Vault
  framework: langgraph
```

Power Automate

One flow per project (or one shared flow that switches on `project`) starting with "When an HTTP request is received". The bot posts JSON; the flow posts an adaptive card to Teams and, for critical incidents, an Outlook email to owners.

The bot only sends when an incident opens, gets worse or resolves, so a bad morning is one message rather than fifty. The morning health report goes through the same flow.

The HTTP trigger needs a Power Automate premium licence. If that isn't available, a Teams "Workflows" webhook works the same way for Teams-only alerts.

```
{
  "project": "slipstream",
  "incident_id": 418,
  "severity": "critical",
  "title": "Site A scrape incomplete",
  "summary": "Rows 88% below Monday
    baseline; price null in 63%...",
  "likely_cause": "Site layout change",
  "action": "Check the price selector
    in the scraper",
  "portal_url": "https://.../p/slipstream/i/418"
}
```

Stack and hosting

PIECE	TECH	RUNS ON
Portal	React + Vite, MSAL for sign-in	Azure Static Web Apps